LLMs Unplugged

Understand AI by building it yourself

Speaker notes

Open with the promise: understand AI by building one yourself, by hand. No computers, no maths background needed.

Acknowledgement of Country

Speaker notes

Acknowledge the Traditional Custodians of the land you're meeting on, and pay respects to Elders past and present. Keep it brief and sincere; say it in your own words.

activity

everyone stand up

Speaker notes

Quick energiser. Read each line and let people sit; it speeds up, and by "the last 5 minutes" almost everyone's down. Lands the point that this stuff is already everywhere. ~2 min, don't linger.

sit down if you

have **never** used ChatGPT

sit down if you

haven’t used it in the last **month**

sit down if you

haven’t used it in the last **week**

sit down if you

haven’t used it in the last **day**

sit down if you

haven’t used it in the last **hour**

sit down if you

haven’t used it in the last **5 minutes**

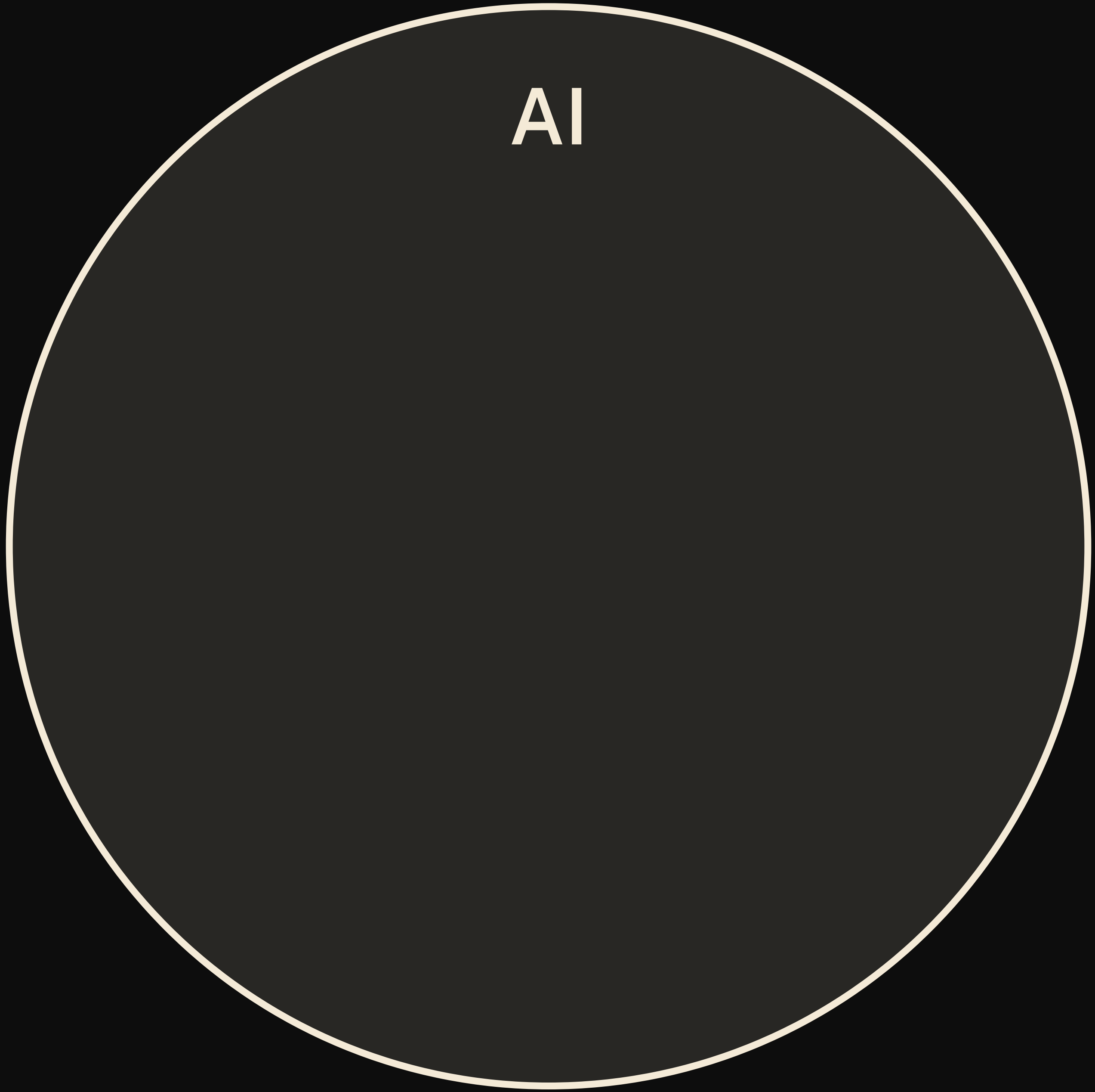
What is this about?

you'll **build** your own language model — from scratch — with just a kids book, pen & paper, and some dice rolling

you'll **learn** how language models work by spotting patterns in text to generate new text

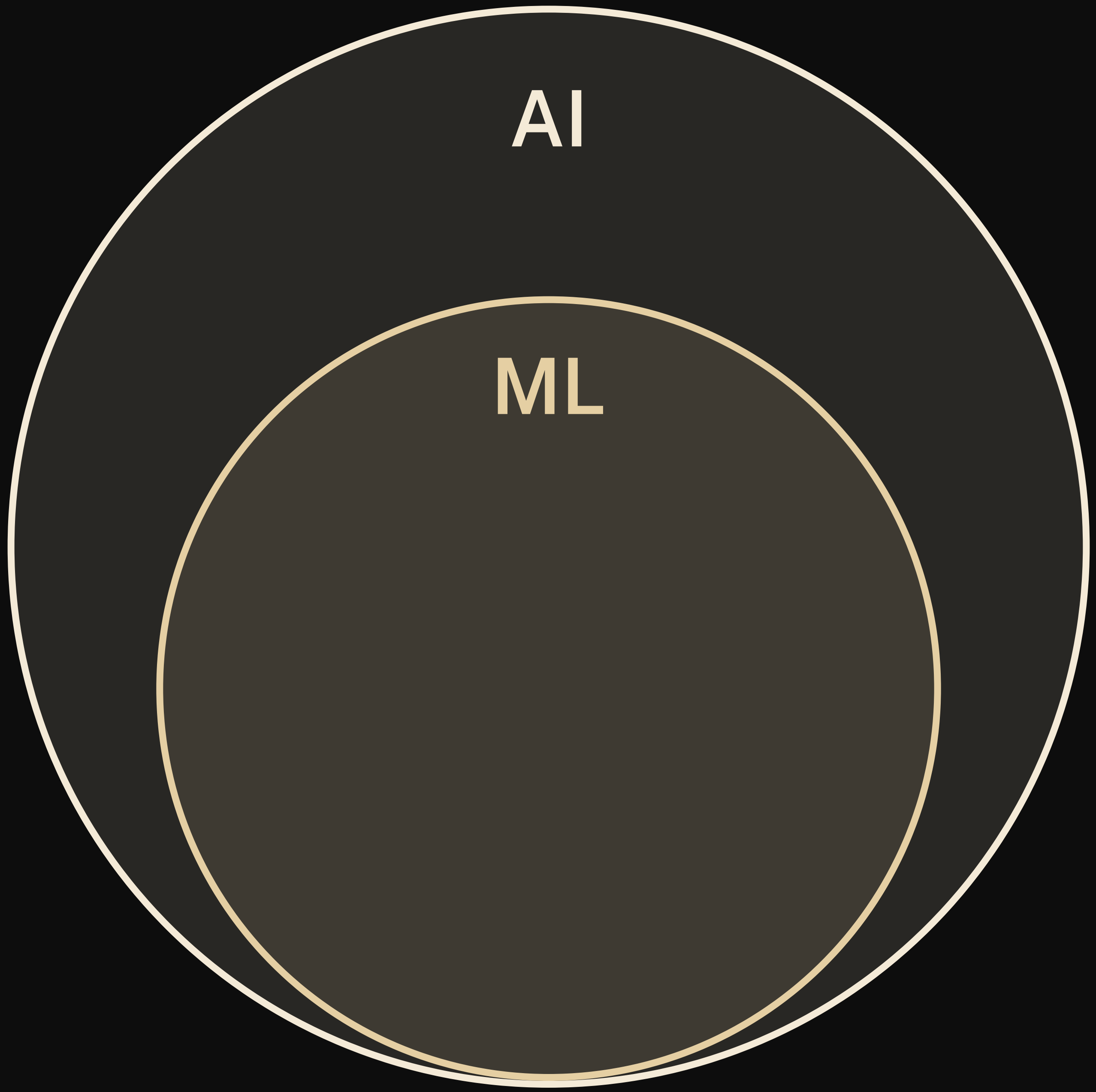
Speaker notes

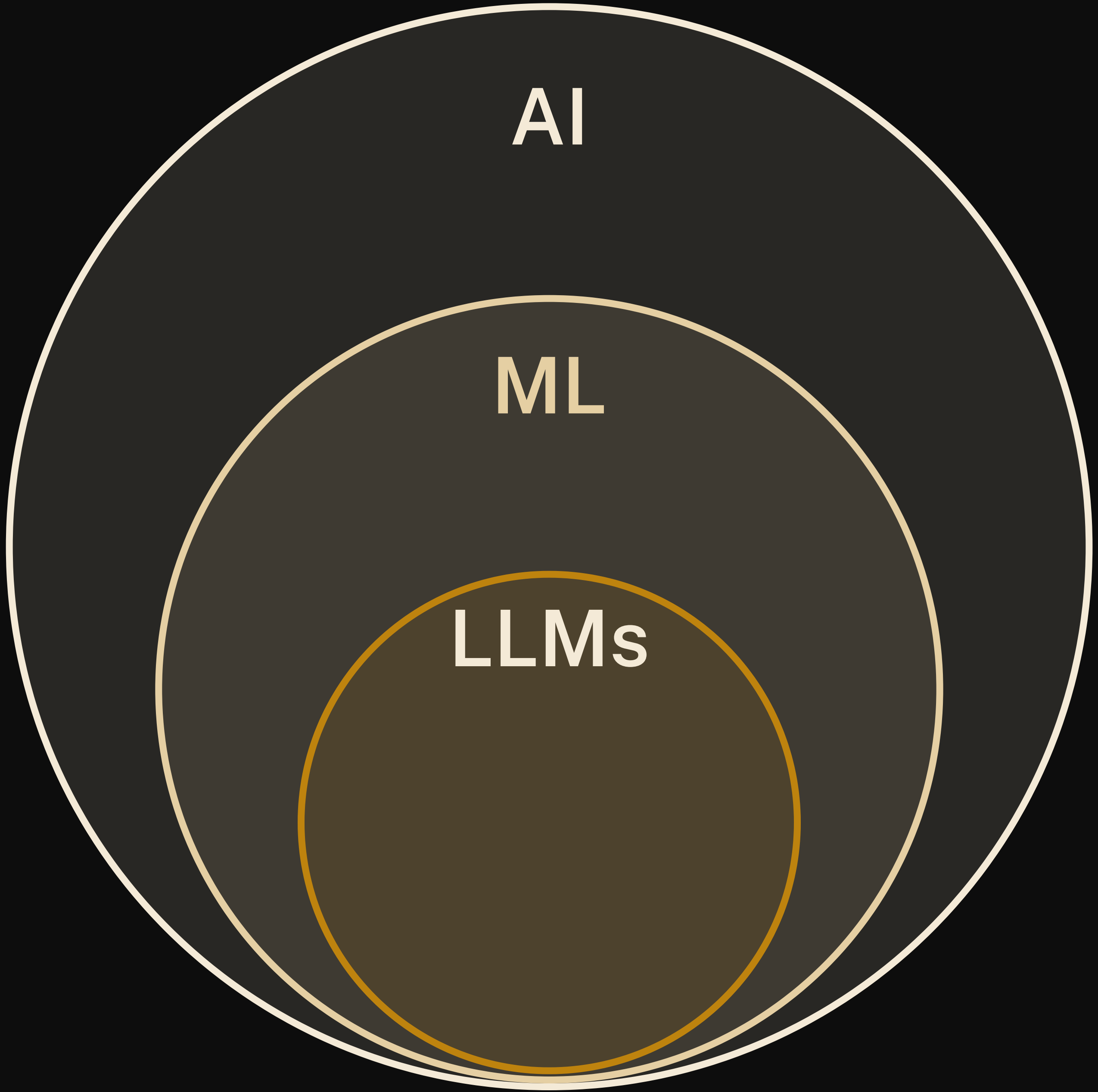
Frame the next stretch: build a model from a kids' book, pen, paper and dice, by spotting patterns. ~1 min, then move.

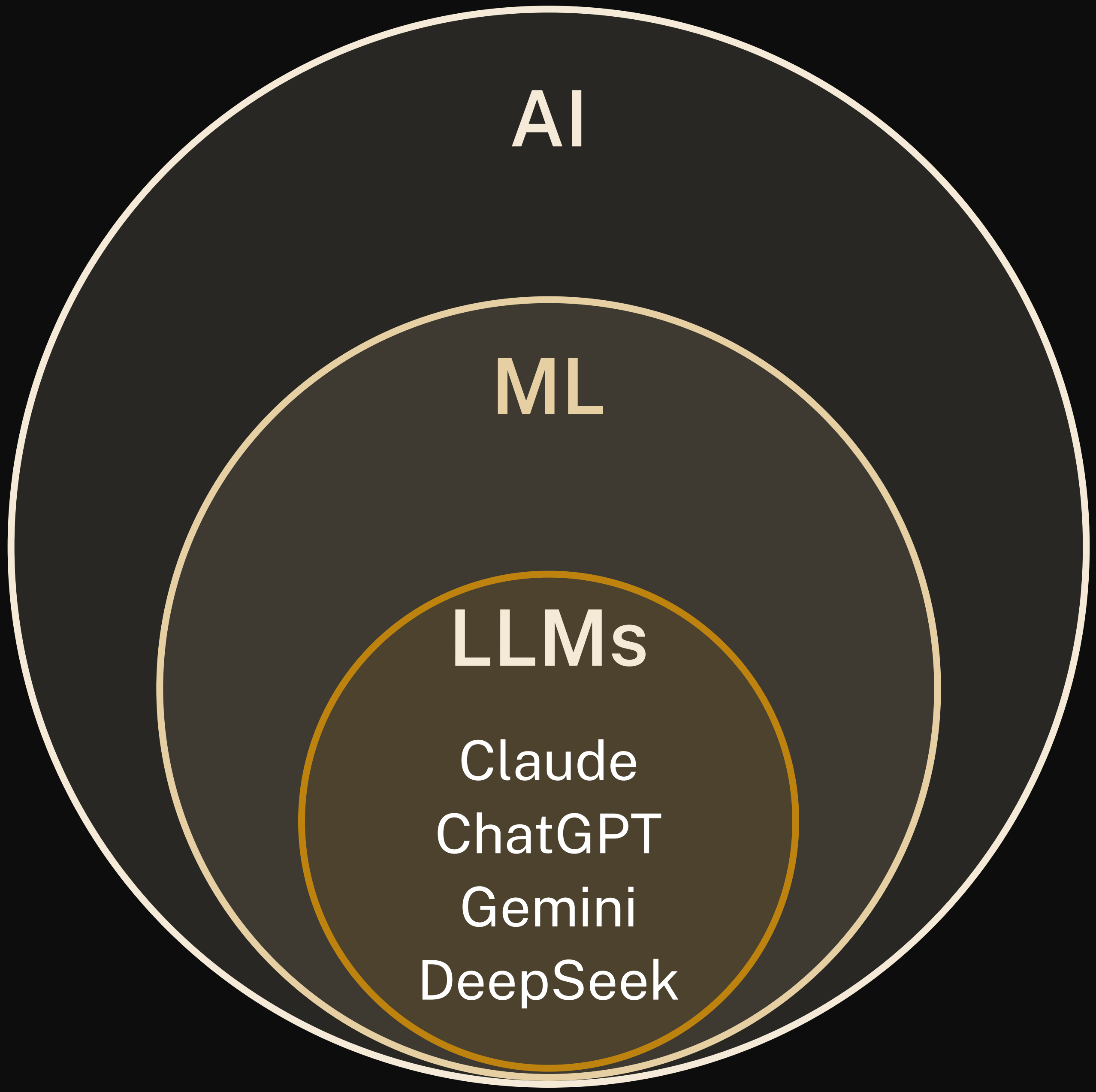

AI

Speaker notes

No text on this one --- talk over the build. AI: the umbrella term, any system doing things we'd call "smart" (chess engines, spam filters, route planners). Click --- ML: the subset that learns patterns from data instead of following hand-written rules; the model you'll build today lives here. Click --- LLMs: machine learning models trained on huge amounts of text to predict the next token --- exactly what you're about to do by hand, just scaled up. Click --- and the chatbots in the news (Claude, ChatGPT, Gemini, DeepSeek) are LLMs wrapped in an app. Same species as the thing you'll build with pen and dice.


AI
ML


AI
ML
LLMs

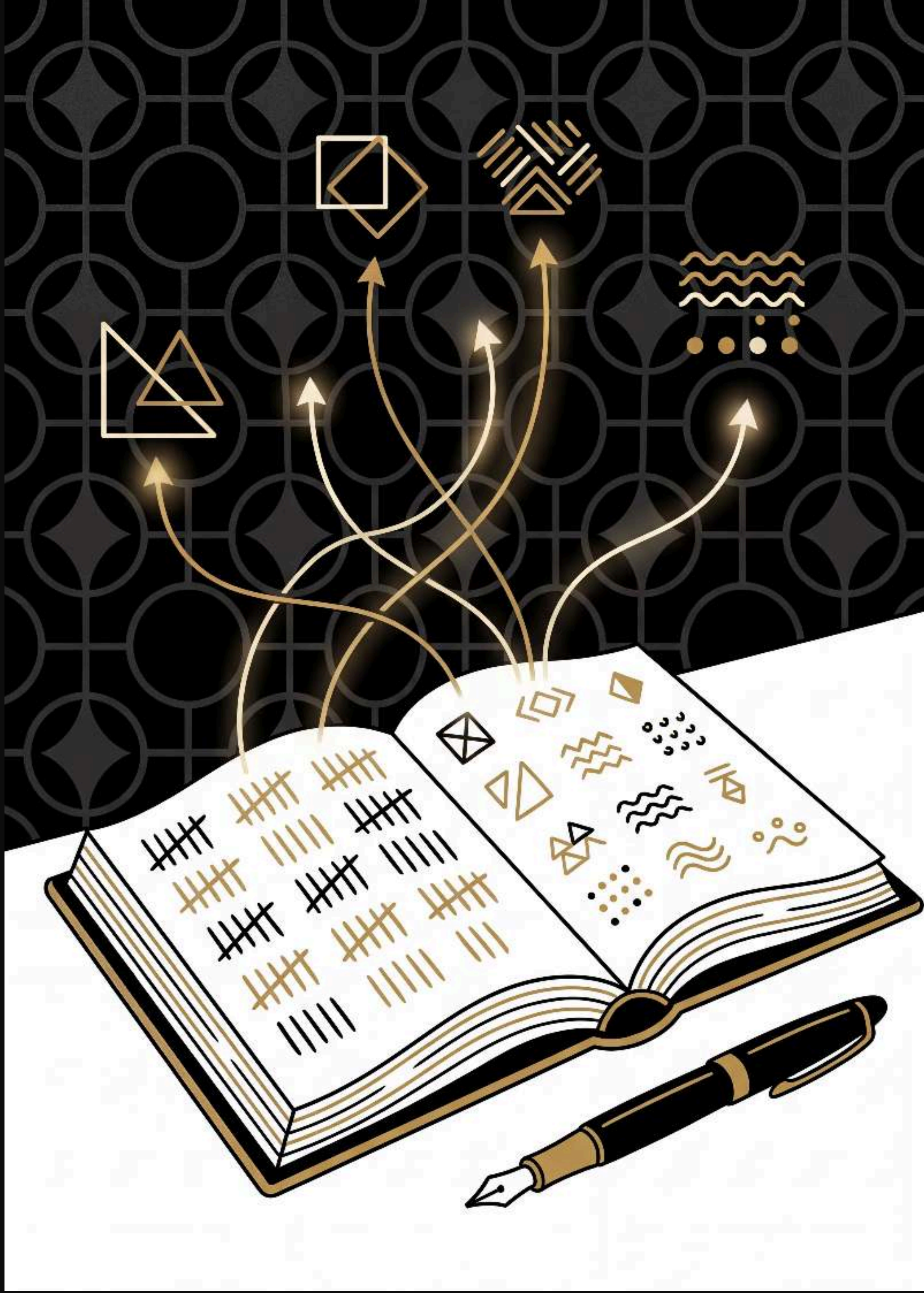

AI
ML
LLMs
Claude
ChatGPT
Gemini
DeepSeek

llmsunplugged.org

Training

The recipe

walk through your text and tally up which tokens follow which in a grid



Speaker notes

1. **tidy up** your text: lowercase everything, treat words and single punctuation marks (`. , ! ? ; :`) as separate tokens
2. **set up** your grid: first token goes in both the row & column headers
3. **fill in** the grid: for each consecutive pair of tokens, add a tally mark in that cell (add new rows/columns as needed)

Example

"Run, Spot, run. See Spot run."

after tidying up:

`run` `,` `spot` `,` `run` `.` `see` `spot` `run` `.`

The empty grid

`run` `,` `spot` `,` `run` `.` `see` `spot` `run` `.`

Training: run → ,

run , spot , run . see spot run .

	run	,			
run					
,					

Training: , → spot

run , spot , run . see spot run .

	run	,	spot		
run					
,					
spot					

Training: spot → ,

run , spot , run . see spot run .

	run	,	spot		
run		|			
,			|		
spot		|			

Speaker notes

`,` is already in our vocabulary — no new column needed!

Training: , → run

run , spot , run . see spot run .

	run	,	spot		
run		|			
,	|		|		
spot		|			

Training: run → .

run , spot , run . see spot run .

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th></th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>|</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td></td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td></td><td></td></tr>
</table>

Speaker notes

`.` is a new token — punctuation gets its own row and column too

Complete model

`run` `,` `spot` `,` `run` `.` `see` `spot` `run` `.`

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

with the rest of the text tallied the same way, the grid is complete. `run` → `.` came up twice, so its cell has two marks — those counts are what make some next-words more likely than others.

Training

10:00

start reset +1 −1

Speaker notes

Hand out grids; groups tally their own text. 10 min, circulate --- the usual snag is that a token only gets a new row and column the first time it appears.

The *language* of language models

- model
- token
- vocabulary
- training

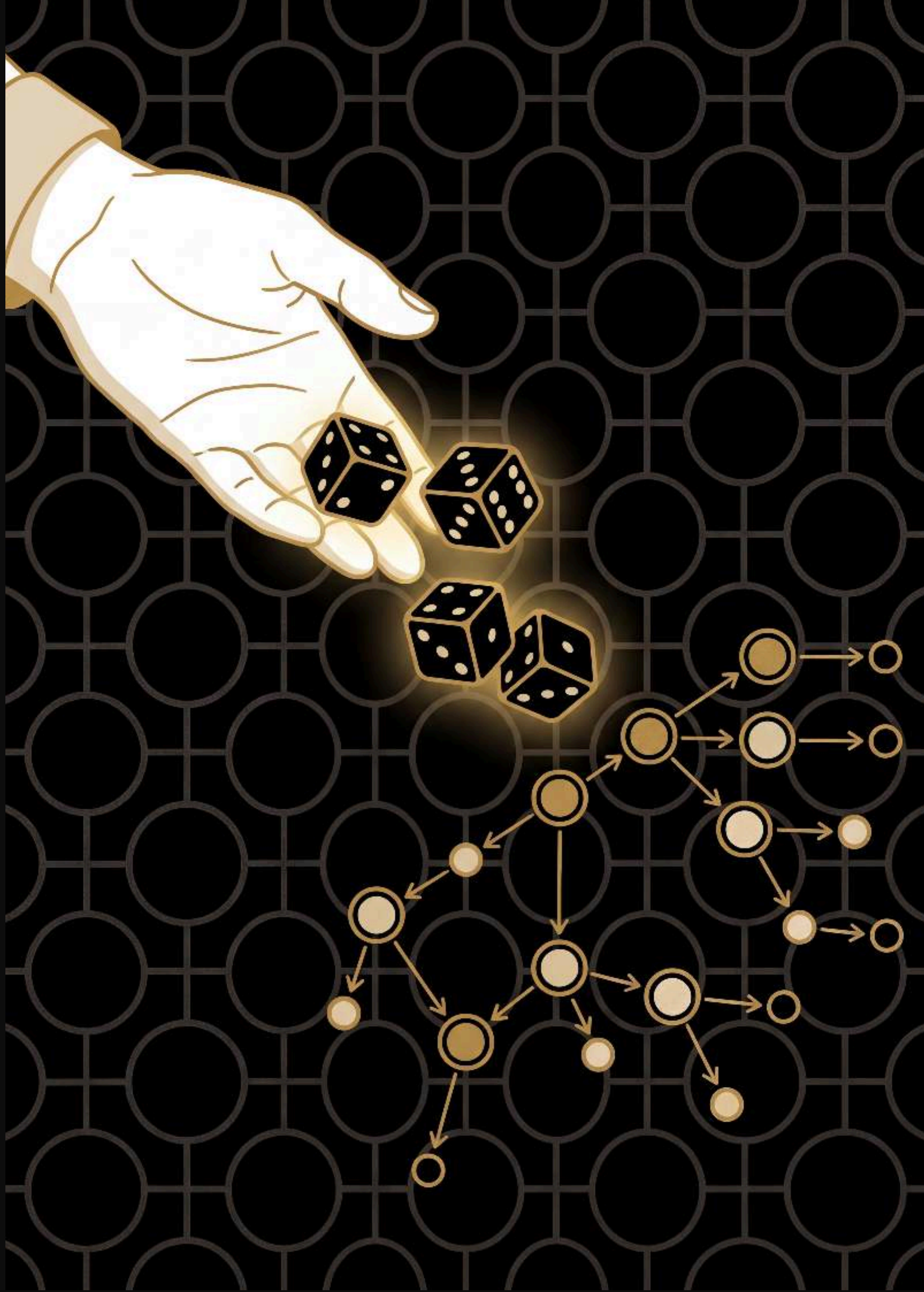
Speaker notes

Gloss each: model = the thing that captures the patterns (your grid); token = one word or punctuation mark; vocabulary = all the distinct tokens it has seen; training = counting which token follows which.

Generation

The recipe

use your grid to generate new text, rolling dice to choose each next word



Speaker notes

1. **choose** a starting word from your grid (any word --- not just the first one)
2. **look** at that word's row to find all possible next words and their counts
3. **roll dice** weighted by the counts to pick the next word
4. **write down** the chosen word and make it your new starting word
5. **repeat** from step 2

Generation: start with see

see spot

one option — no roll needed

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

any word will do as a seed---we're starting from `see`, which is the last row, to show you don't have to begin at the top. Only one option here, so no roll.

Generation: from `spot`

`see` `spot`

2 options — roll the die!

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

two options in this row (`run` and `,`) with equal tallies---highlight both, then roll

How the die chooses: `spot`

`spot` → ? roll a d10

0	1	2	3	4		5	6	7	8	9

`run`
1 tally

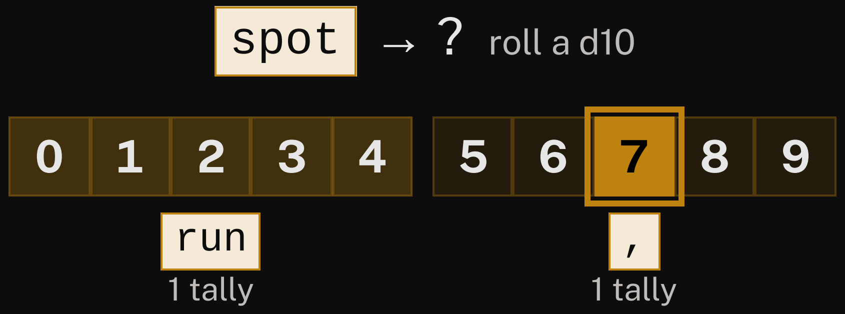
`,`
1 tally

equal tallies → equal chances

Speaker notes

equal tallies, so each option gets an equal share of the ten d10 faces --- `run` gets 0-4 and `,` gets 5-9, reading the row's cells left to right. Pause here before rolling.

How the die chooses: `spot`



equal tallies → equal chances

rolled 7 → `,`

Speaker notes

now roll --- a 7 lands in the `,` band

Generation: spot → ,

see spot ,

rolled 7 → ,

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

the 7 landed in the `,` band, so `,` is our next word

Generation: from ,

see spot ,

2 options — roll the die!

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

another two-option row (`run` and `spot`), still equal --- both highlighted, then roll

Generation: , → run

see spot , run

rolled **2** → run

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

rolled a 2, which lands in the `run` band

Generation: from `run`

`see` `spot` `,` `run`

2 options — roll the die!

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

two options again --- but `.` has two tallies to `,`'s one, so this isn't a 50/50 roll

How the die chooses: `run`

`run` → ? roll a d10

0	1	2	3	4	5	6	7	8	9

`,` 1 tally

`.` 2 tallies

more tallies → more faces → more likely

Speaker notes

`.` has twice the tallies, so it gets more of the faces (3-9) than `,` (0-2). With a ten-sided die that's the closest split rather than exactly 2:1---the point is more tallies → more faces → more likely. Pause here before rolling.

How the die chooses: `run`

`run` → ? roll a d10

0	1	2	**3**	4	5	6	7	8	9

`,` 1 tally

`.` 2 tallies

more tallies → more faces → more likely

rolled **3** → `.`

Speaker notes

now roll---a 3 sits in the `.` band

Generation: run → .

see spot , run .

rolled **3** → .

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

the 3 sits in the `.` band, so we pick `.`

Generation: from `.`

`see` `spot` `,` `run` `.` `see`

one option — no roll needed

	run	,	spot	.	see
run		ǀ		ǁ	
,	ǀ		ǀ		
spot	ǀ	ǀ			
.					ǀ
see			ǀ		

Speaker notes

only one option (`see`) --- no roll. Note we keep rolling straight past the full stop; the model has no idea a sentence just ended.

Generation: back to see

see spot , run . see

one option — no roll needed

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

and so on --- we've looped back to where we started. Keep going for as long as you like; you decide when to stop.

Generated text

"see spot, run. see"

a new sentence — not in the training data!

Generation

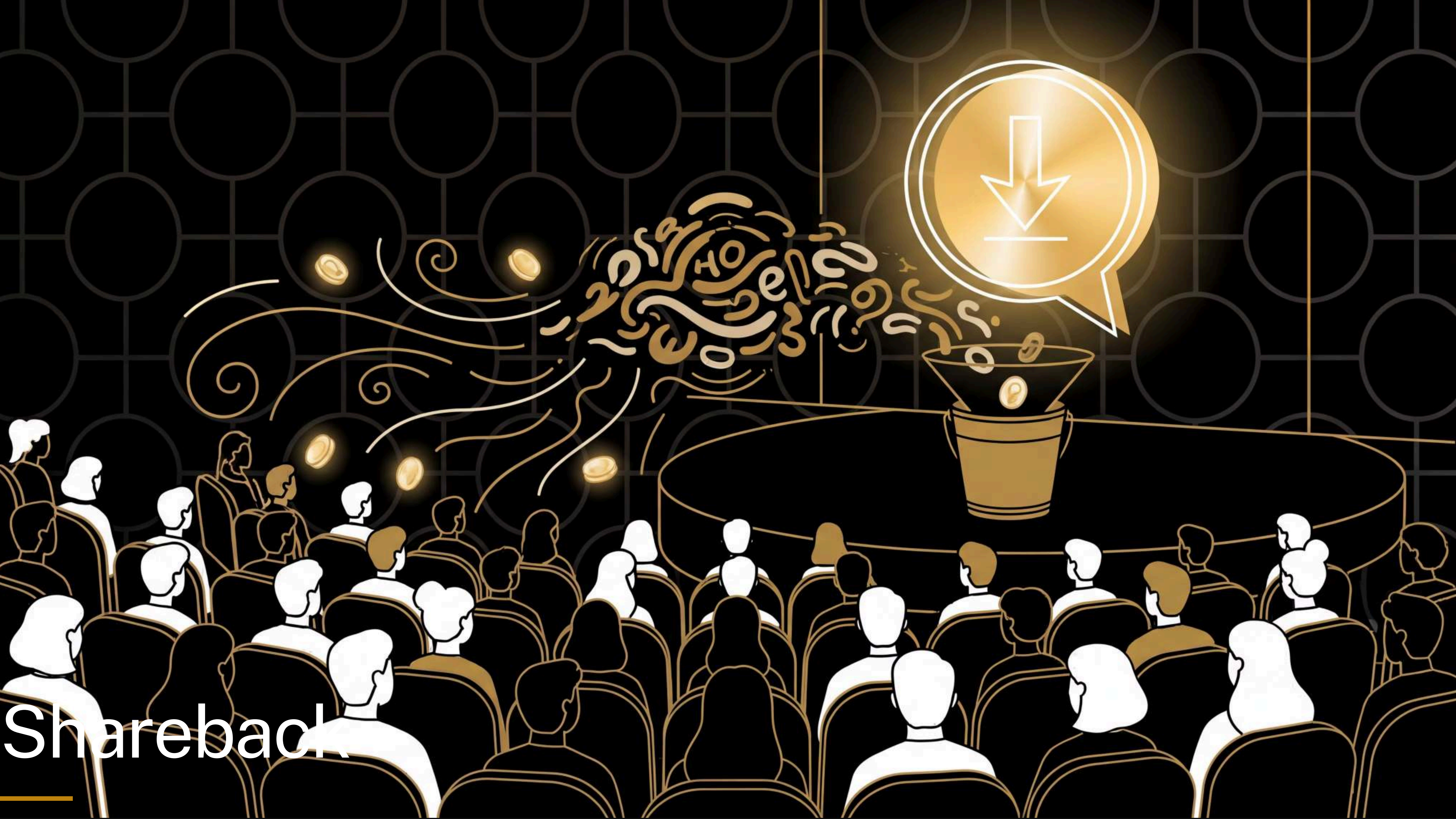
10:00

start reset +1 −1

Speaker notes

Groups generate from their grid, rolling dice on each choice. 10 min, circulate. Where a row has only one option, no roll needed.

Shareback



Speaker notes

Take a couple of groups' generated sentences; celebrate the weird ones. ~5 min.

The *language* of language models

- prompt
- response/completion
- context window

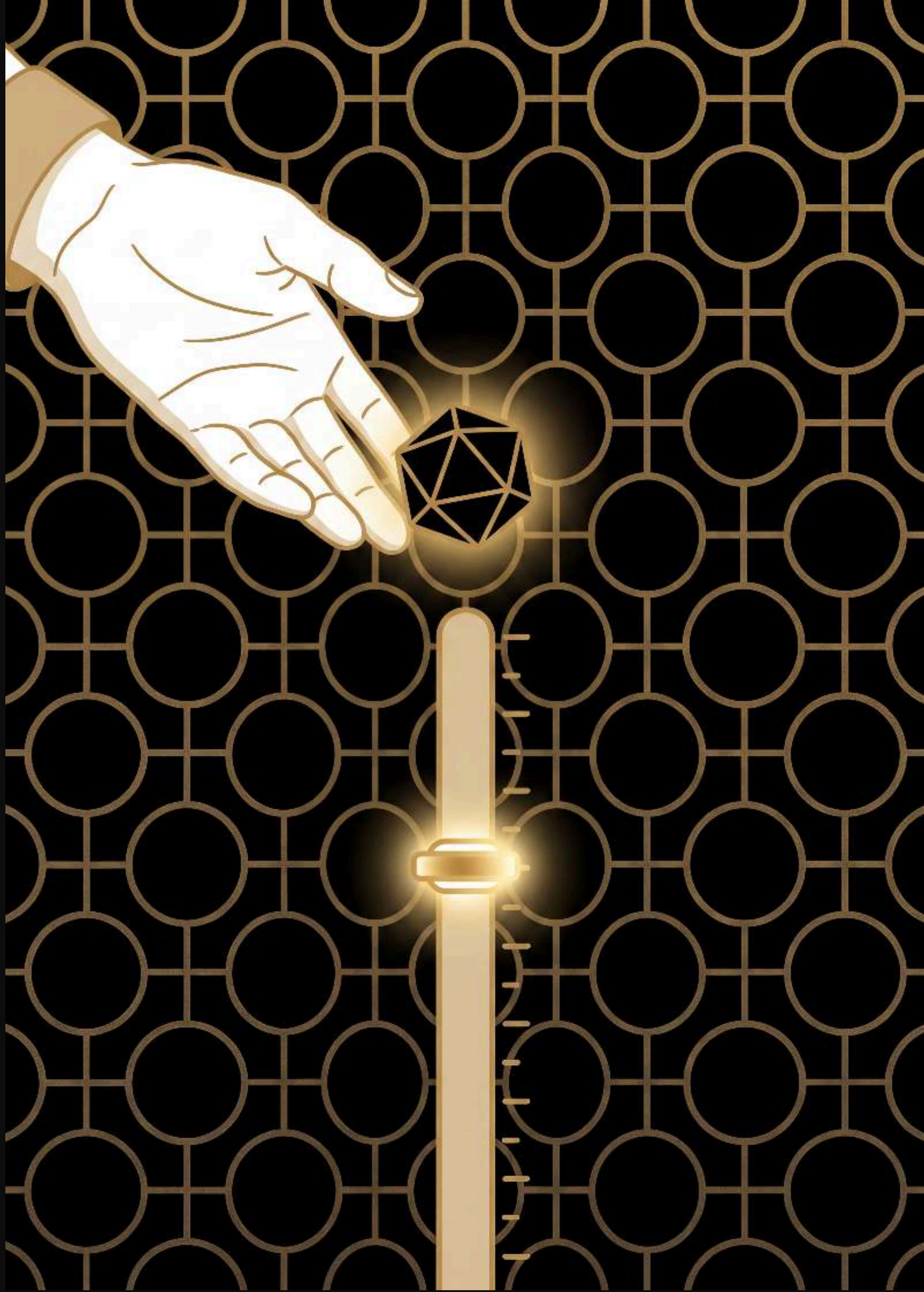
Speaker notes

Gloss each: prompt = the starting text you give it; response/completion = the text it generates back; context window = how much it can "see" at once (here, just the current word).

Pre-trained generation

The recipe

use the booklet to generate new text, rolling one or two d10s to choose each next word



Speaker notes

1. **choose** a starting word---any bold word in the booklet
2. **look up** that word's entry to see possible next words and their thresholds
3. **roll** your d10(s) and scan the thresholds to find the next word
4. **write down** the chosen word and make it your new starting word
5. **repeat** from step 2

Pre-trained: start with `the`

`the` `cat`

the ♦♦ 49|cat 79|dog 99|hat **rolled 27**

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

a bigger model — `the` is so common it needs **two** dice (roll two d10s for 00 to 99)

Pre-trained: from `cat`

`the` `cat` `sat`

the ♦♦ **49**|cat **79**|dog **99**|hat

cat ♦ **5**|sat **9**|ran **rolled 4**

sat .

. the

ran .

dog ♦ **6**|sat **9**|ran

hat ♦ **4**|sat **9**|ran

Speaker notes

`cat` is rarer than `the` — just **one** die, roll a d10

Pre-trained: from `sat`

`the` `cat` `sat` `.`

the ♦♦ **49**|cat **79**|dog **99**|hat

cat ♦ **5**|sat **9**|ran

sat .

. the

ran .

dog ♦ **6**|sat **9**|ran

hat ♦ **4**|sat **9**|ran

Speaker notes

only one option — no dice roll needed

Pre-trained: from .

the cat sat . the

the ♦♦ 49|cat 79|dog 99|hat

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

only one option — no dice roll needed

Pre-trained: from the again

the cat sat . the dog

the ♦♦ 49|cat 79|dog 99|hat **rolled 63**

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

two dice again — 63 is past `cat`'s 49, so we land on `dog`

Pre-trained: from dog

the cat sat . the dog

the ♦♦ 49|cat 79|dog 99|hat

cat ♦ 5|sat 9|ran

sat .

. the

ran .

dog ♦ 6|sat 9|ran

hat ♦ 4|sat 9|ran

Speaker notes

and so on...

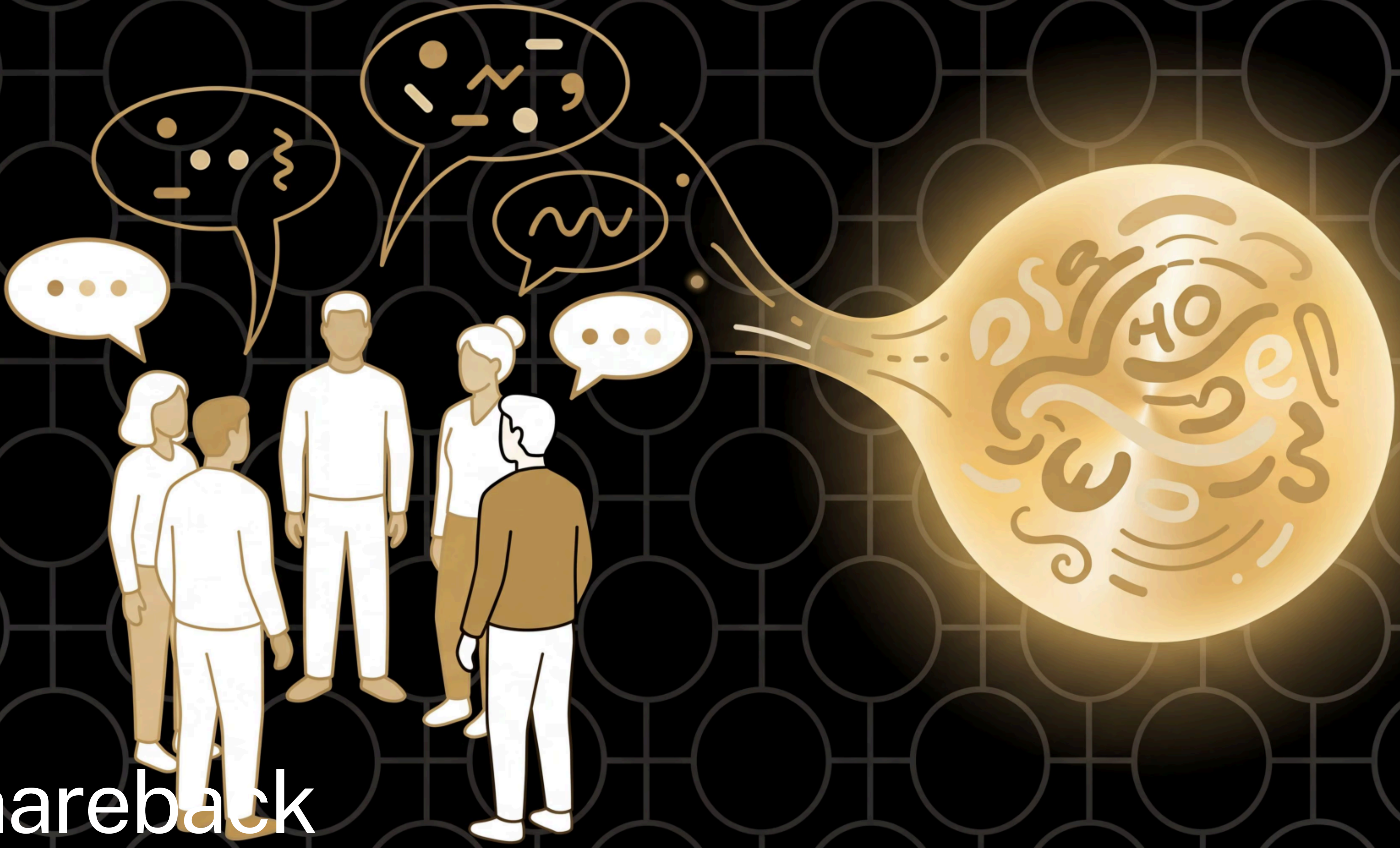
Pre-trained generation

8:00

start reset +1 −1

Speaker notes

Same as before but from the booklet --- a bigger model, one or two d10s. 8 min, circulate.



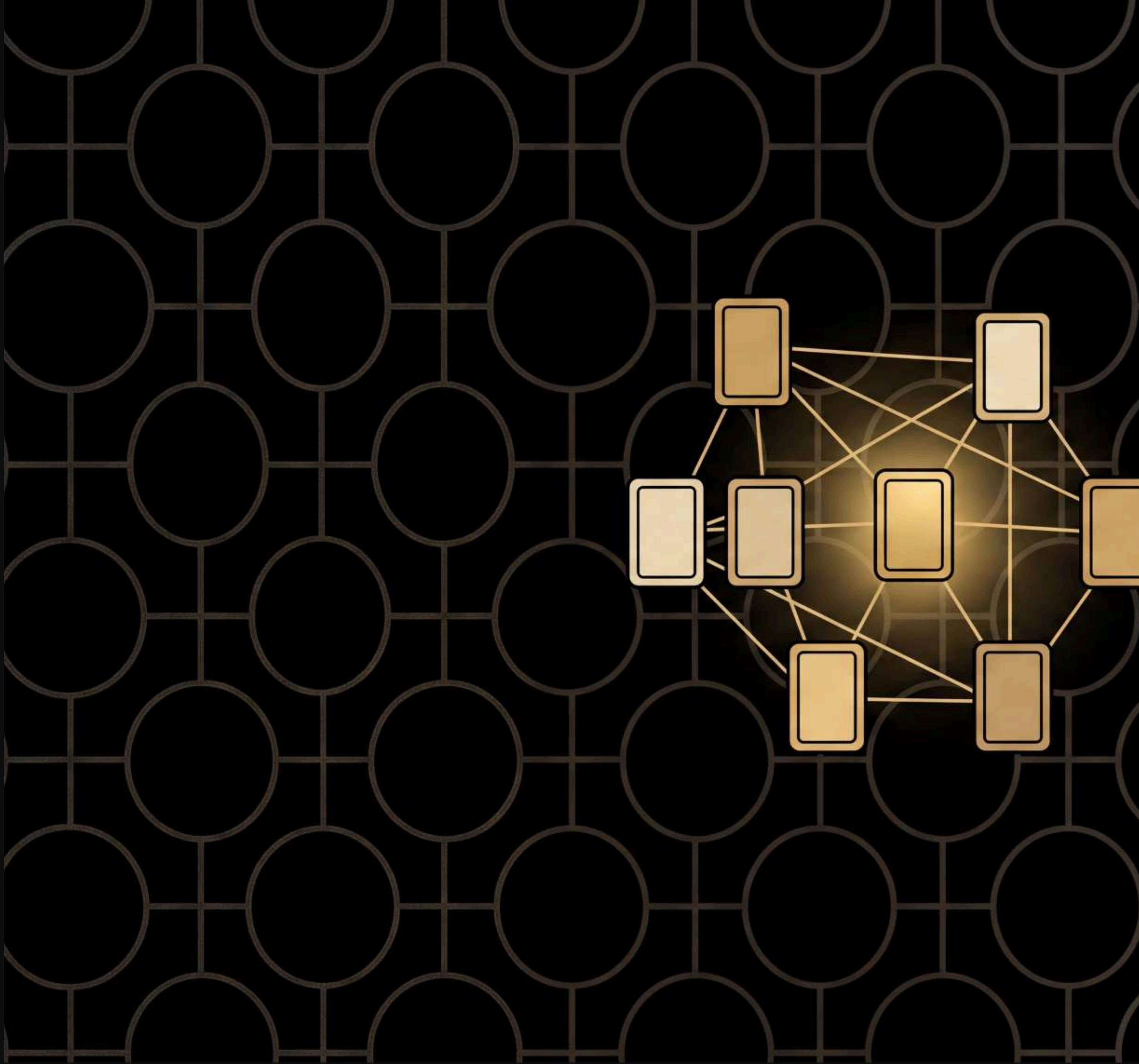
Shareback

Speaker notes

Compare with the hand-built model: a bigger training text gives more varied, more natural output. Invite ~5 groups to share back rather than the whole room. ~5 min.

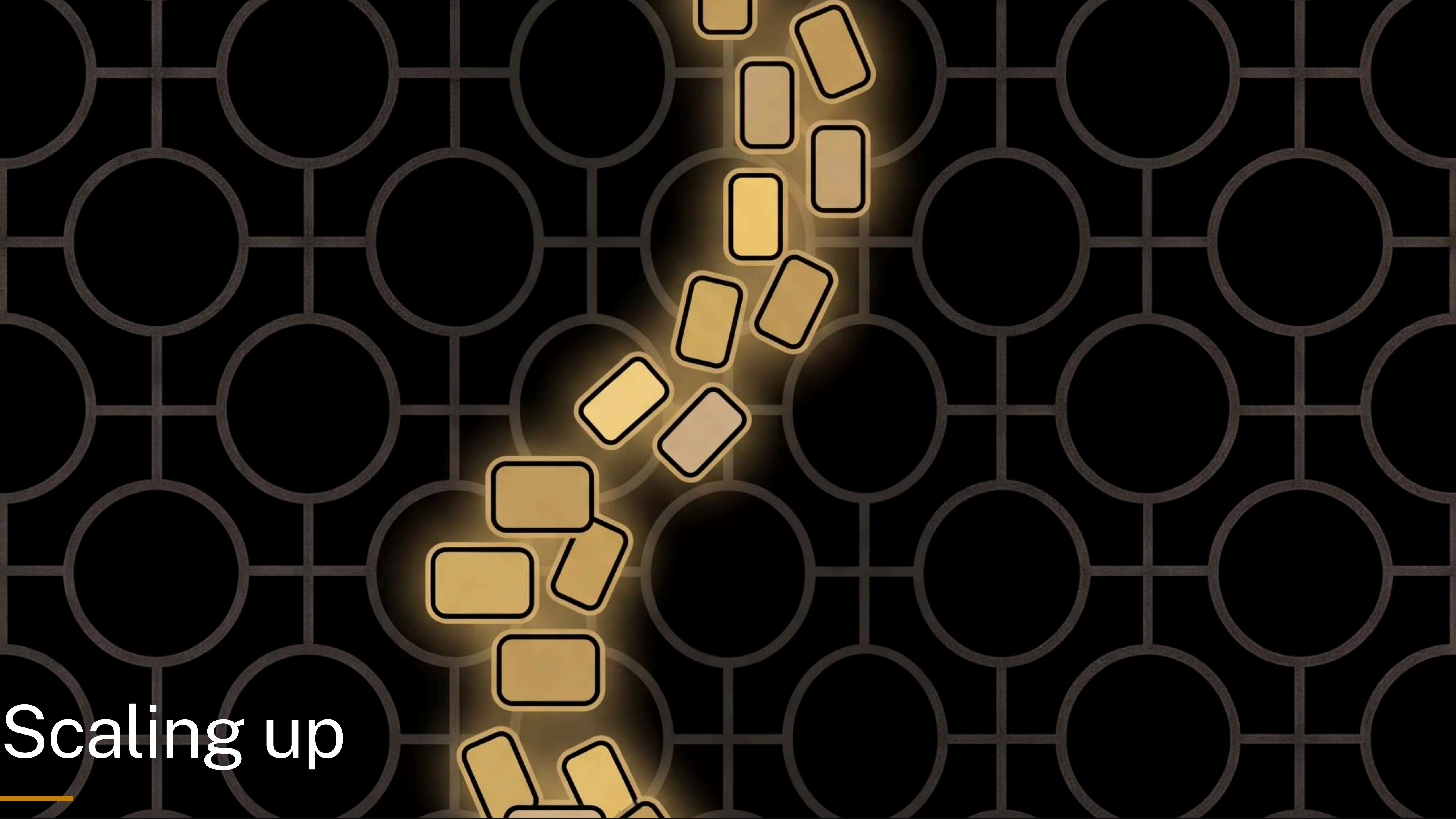
The *language* of language models

- pre-training
- foundation model



Speaker notes

Gloss each: pre-training = the big training run done ahead of time on huge amounts of text; foundation model = the general-purpose model that run produces, which other things are built on.



Scaling up

Speaker notes

The reassurance beat: what they built isn't a cartoon of an LLM, it's the real mechanism at human scale --- look at the context, weigh every possible next word, roll the dice, write it down, repeat. So what's actually different? The whole section is run against their own grid rather than as a set of separate diagrams. Four times over, the grid is made to fail at something on stage, and the thing that fixes it turns out to be a real part of a transformer: more context (and why the grid can't have it), attention, parameters, post-training. Then scale. ~12 min --- it's a lot of slides but most are one beat each, so keep moving.

One word of context

see spot ,

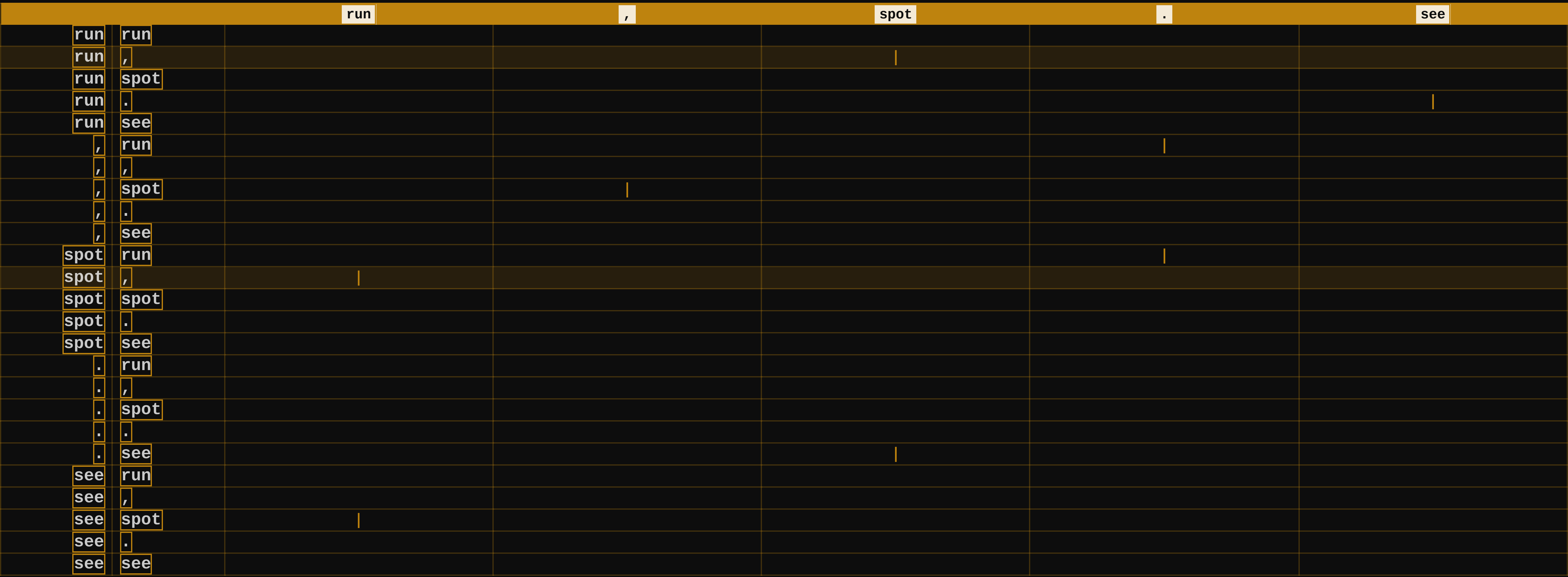
2 options — roll the die!

	run	,	spot	.	see
run		|		||	
,	|		|		
spot	|	|			
.					|
see			|		

Speaker notes

Back to the worked example. We're sitting on a comma with two options and no way to choose between them---so we roll. But look at why we're stuck: the model is looking at a comma, and every comma in the text looks identical to it. It cannot tell whether it just wrote `run` or `spot`. That's not bad luck, it's the whole design: one word of context.

Two words of context



so remembering two words instead of one fixes it — what does that cost?

Speaker notes

Give it two words and the guess stops being a guess. `run ,` is always followed by `spot`; `spot ,` is always followed by `run`. Same text, same tallies, no dice needed --- the ambiguity was never in the language, it was in how little the model could see. This is a trigram model, and there's a lesson on the website that runs it with pen and paper.

Every context needs its own row

one word: 5 rows

two words of context

25 rows, one per possible pair

and almost every one of them is empty

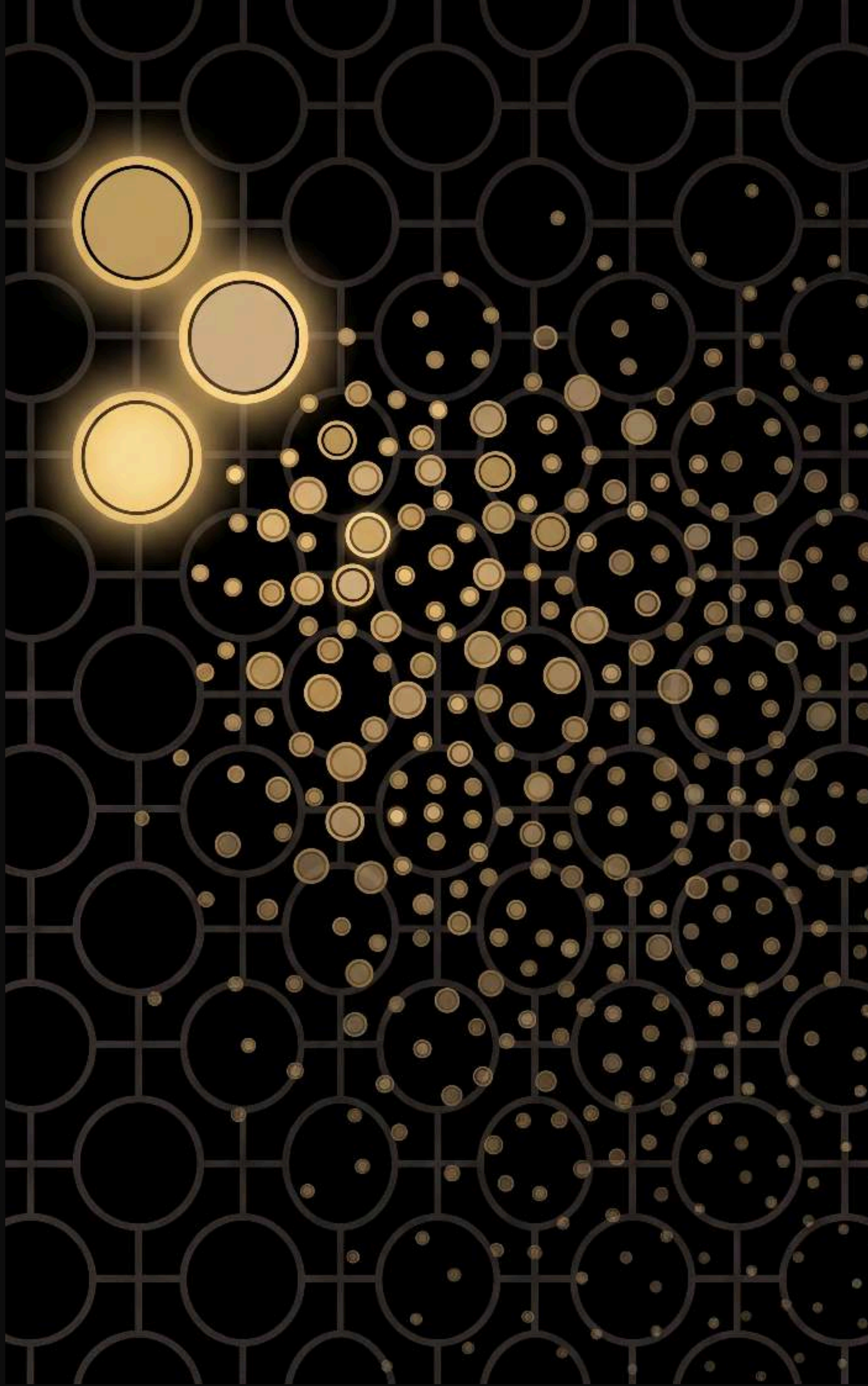
Speaker notes

This is the bill. The grid needs a row for every context it might find itself in, so going from one word to two takes 5 rows to 25 --- and most of them stay empty, because our text never contains `see see` or `. .`. Three words would be 125. Each extra word of memory multiplies the grid by the size of the vocabulary.

The grid nobody can build

context	rows in the grid
1 word	5
2 words	25
3 words	125
10 words	9,765,625

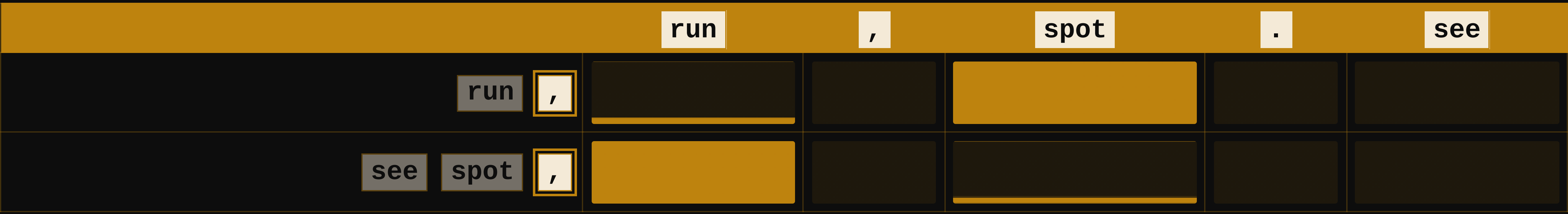
Claude keeps **200,000 words** in view, over a vocabulary of **100,000**



Speaker notes

Read the table, then drop the last line. Their vocabulary is 5, so it's 5-to-the-n. A real model's vocabulary is around 100,000, which is 10-to-the-5 --- so seventeen words of context is already 10-to-the-85 rows, and the observable universe only has about 10-to-the-80 atoms. Seventeen words. Claude holds two hundred thousand. The grid is not big, it is impossible, and that's not rhetoric --- you run out of universe long before you run out of sentence. So something else has to be going on.

Attention: a row that changes shape



one row, re-weighted by what came before — instead of a row per context

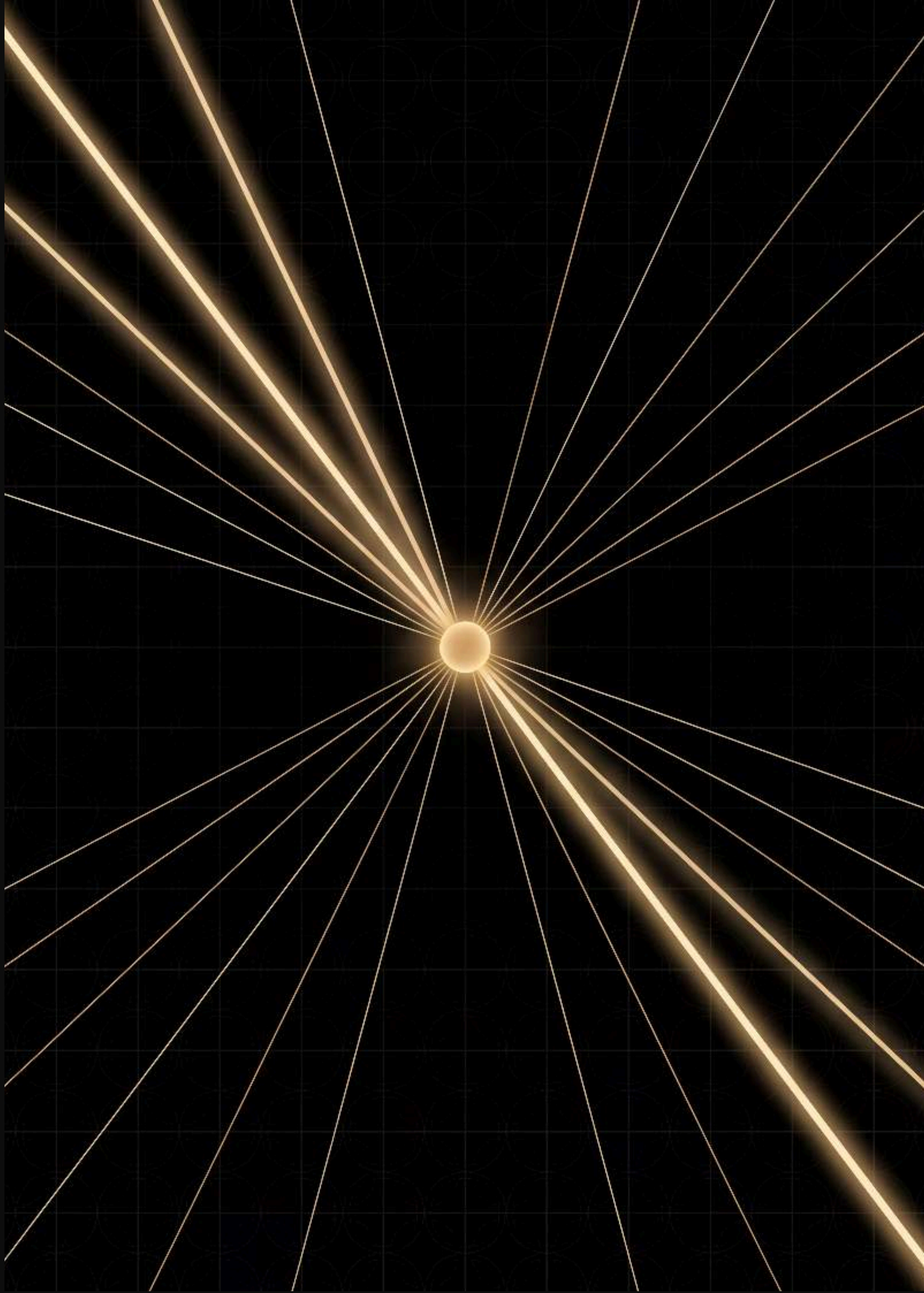
Speaker notes

Here's the trick that replaces the impossible grid. Rather than storing a separate row for every context, the model keeps one row and re-shapes it on the fly: it looks back over everything it can see and weighs which earlier words matter for this prediction. Same comma, same row, different answer, because `see spot` before it pulls the weight one way and `run` pulls it the other. That's attention, and it's why transformers can have a context window at all.

Attention: learning where to look

`the` `wifi` `password` `is` `swordfish` ··· `the` `password` `is` `?`

the next word is *swordfish* — attention finds where `password` last appeared and copies what came after



Speaker notes

A concrete one to make it stick. The model spots the earlier "the password is swordfish" and repeats what followed. That look-back-and-copy move is an induction head, and the Induction Heads lesson on the website is exactly this pattern --- "find the last time I saw this word, copy what came next" --- run with pen and paper. The weights aren't hand-set either: the model learns where to look.

You can point at the pattern

see spot , run .

rolled **3** → .

<table>
<tr><th></th><th>run</th><th>,</th><th>spot</th><th>.</th><th>see</th></tr>
<tr><td>run</td><td></td><td>|</td><td></td><td>||</td><td></td></tr>
<tr><td>,</td><td>|</td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>spot</td><td>|</td><td>|</td><td></td><td></td><td></td></tr>
<tr><td>.</td><td></td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>see</td><td></td><td></td><td>|</td><td></td><td></td></tr>
</table>

Speaker notes

Second failure, and it starts with something the grid is good at. `run` is followed by a full stop twice, and there are the two marks, in that cell, and you can put your finger on them. Everything the model knows is somewhere you can point at. Hold that thought.

cat and dog are strangers

the cat sat . the cat ran . the dog sat .

<table>
<tr><th></th><th>the</th><th>cat</th><th>sat</th><th>.</th><th>ran</th><th>dog</th></tr>
<tr><td>the</td><td></td><td>||</td><td></td><td></td><td></td><td>|</td></tr>
<tr><td>cat</td><td></td><td></td><td>|</td><td></td><td>|</td><td></td></tr>
<tr><td>sat</td><td></td><td></td><td></td><td>||</td><td></td><td></td></tr>
<tr><td>.</td><td>||</td><td></td><td></td><td></td><td></td><td></td></tr>
<tr><td>ran</td><td></td><td></td><td></td><td>|</td><td></td><td></td></tr>
<tr><td>dog</td><td></td><td></td><td>|</td><td></td><td></td><td></td></tr>
</table>

Speaker notes

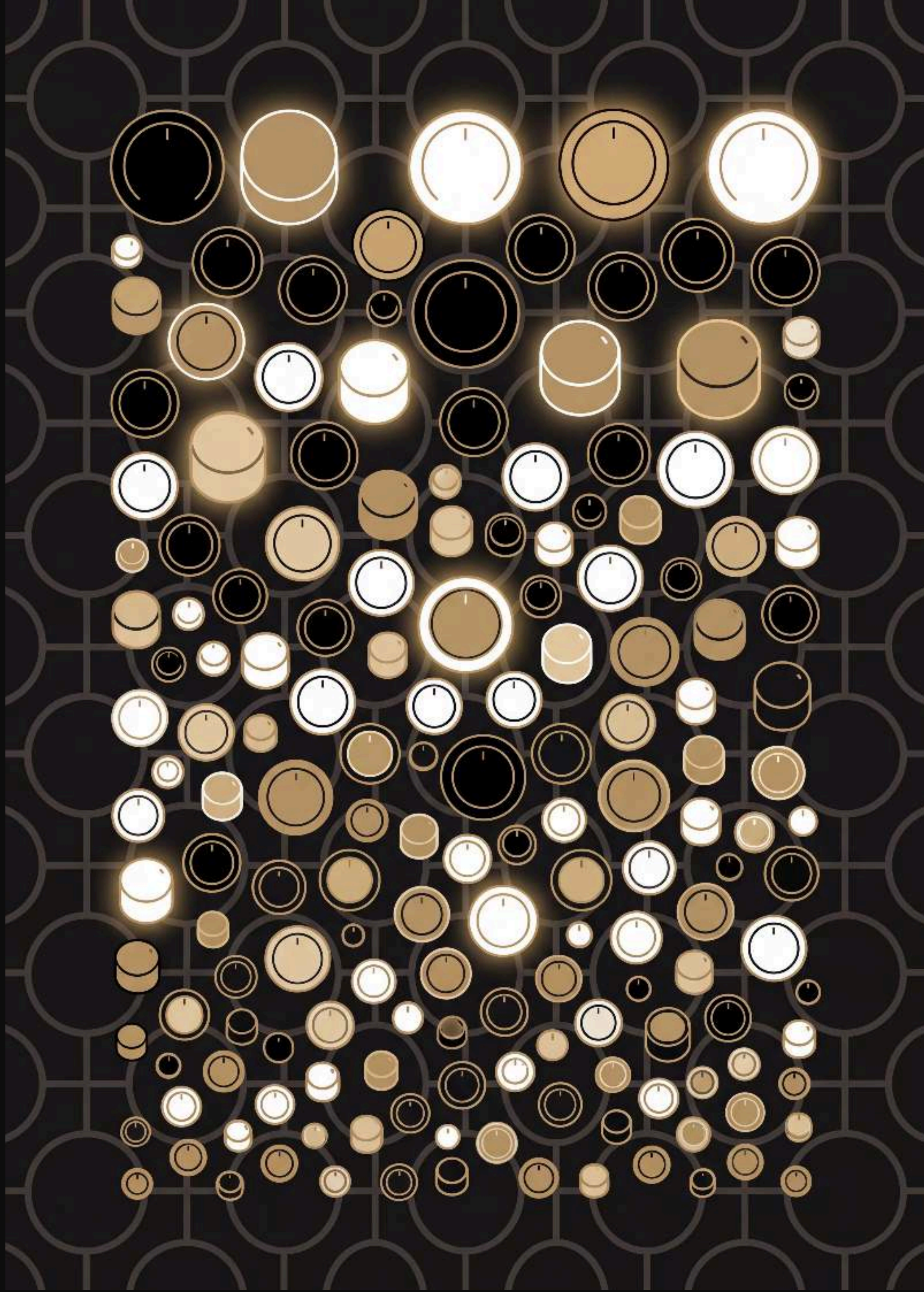
A different text, same idea. `cat` and `dog` do exactly the same job here---both come after `the`, both get followed by verbs. But look at the two rows: `cat` has seen `sat` and `ran`; `dog` has only seen `sat`. Ask this grid to continue "the dog" and it will never, ever say `ran`, because that cell is empty and an empty cell means impossible. You know "the dog ran" is fine. The grid has no way to know, because nothing it learned about `cat` is connected to `dog` in any way. Every cell is its own little island.

Tallies become knobs

your grid `run` `.` | |

a transformer `0.31` `-1.20` `0.08` `0.94` `-0.55` `1.30` × *billions*

no cell for `dog` — just **billions of numbers** that `cat` and `dog` both help set, and both draw on



Speaker notes

The fix for the island problem. A transformer doesn't give each pattern its own cell; it spreads every pattern across billions of numbers, each nudged a tiny amount after every wrong guess in training. Because `cat` and `dog` turn up in the same kinds of places, training pushes them onto overlapping numbers --- so what the model learns from `cat` is already, for free, something it knows about `dog`. That's generalisation, and it's the thing the grid structurally cannot do. The cost is the flip side of the slide before: there is no longer a cell to point at. Ask where a transformer stores "the dog ran" and the honest answer is "smeared across a few billion numbers, and we're still working on reading it back". That's most of what interpretability research is.

Ask your grid a question

prompt: “what is the capital of France?”

your grid `“see spot, run. see”`

it isn’t refusing to answer — **answering isn’t a thing it does**

Speaker notes

Third failure, and the funniest one. Feed their model a question and it does what it always does: continues. It isn't being unhelpful and it isn't broken; nothing in "tally which word follows which" has any notion of a question and an answer. This is the honest state of a model that has only ever been trained to continue text.

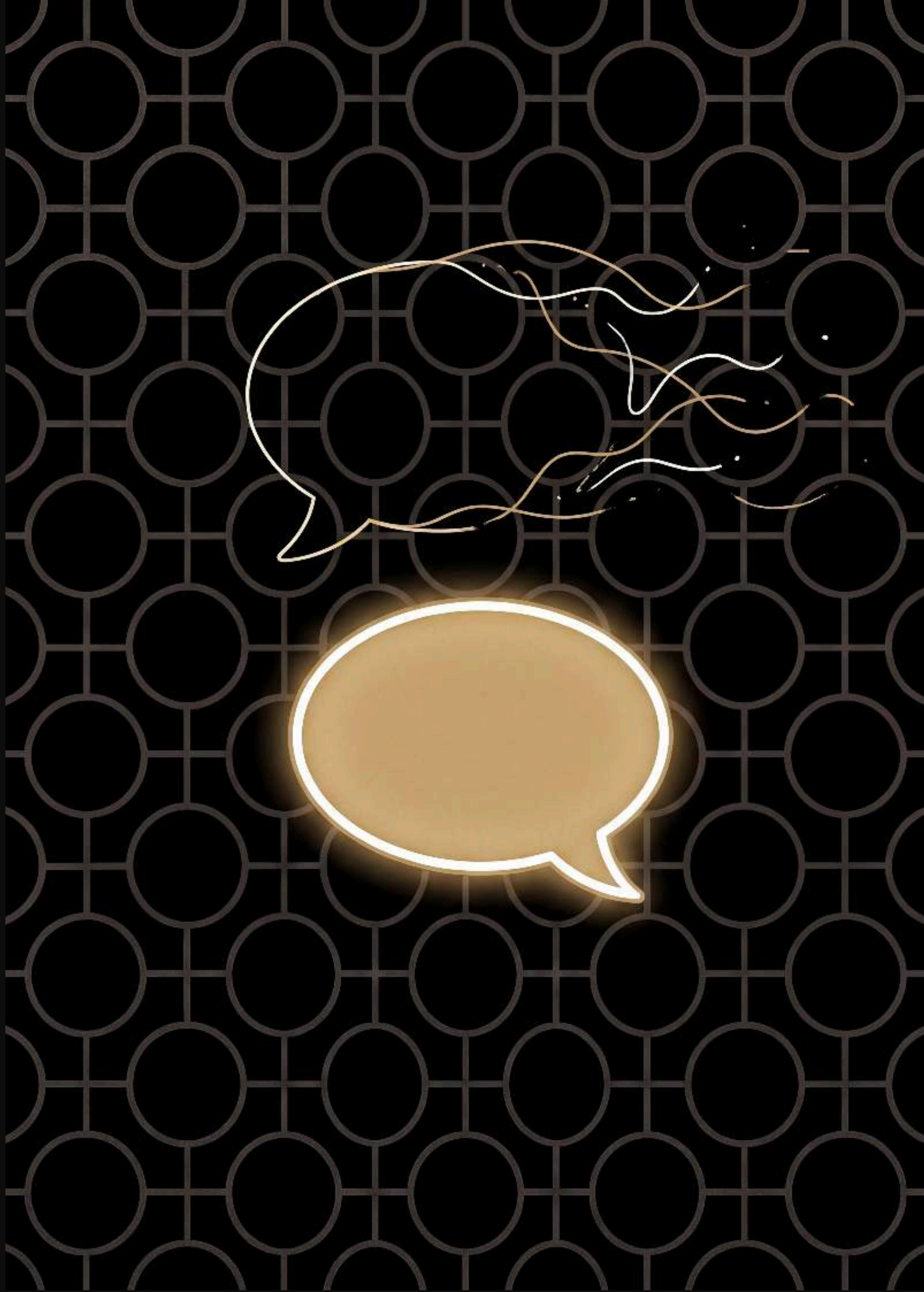
From continuing to answering

prompt: "what is the capital of France?"

a base model — same problem, more fluent `"What is the capital of Germany? What is the largest…"`

+ post-training `"Paris."`

post-training: the same tallying again, over text made of *conversations*



Speaker notes

And a frontier model straight out of pre-training has exactly their problem, just with better grammar --- prompt it with a question and the plausible continuation is more questions. The chat behaviour comes from a second, much smaller round of training on example conversations, plus feedback on which answers people preferred. Worth being clear: this is not a different mechanism. Same predict-the-next-word loop, same dice; a different text to learn from, so different habits on top.

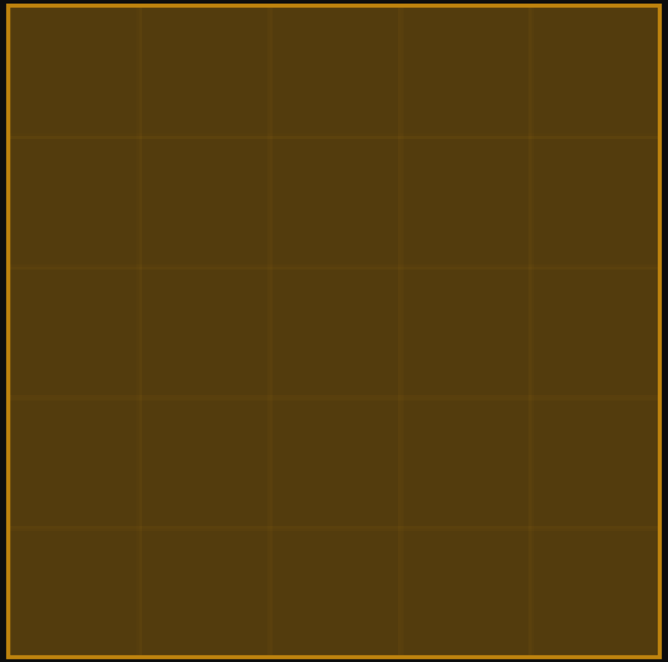
Scale is a hell of a drug

Speaker notes

Four things fixed. None of them changed the loop. The last difference is just dosage --- and it's worth taking slowly, because this is the one people's intuition has no grip on. Keep your eye on the gold square: that's their grid, drawn at true scale, on every slide.

Your grid

your grid



“Run, Spot, run. See Spot run.”

5 tokens · 25 cells

Speaker notes

Here it is: five tokens, twenty-five cells. At the booklet's 1cm per cell that's a five-centimetre square --- smaller than a beer coaster.

Green Eggs and Ham

your grid

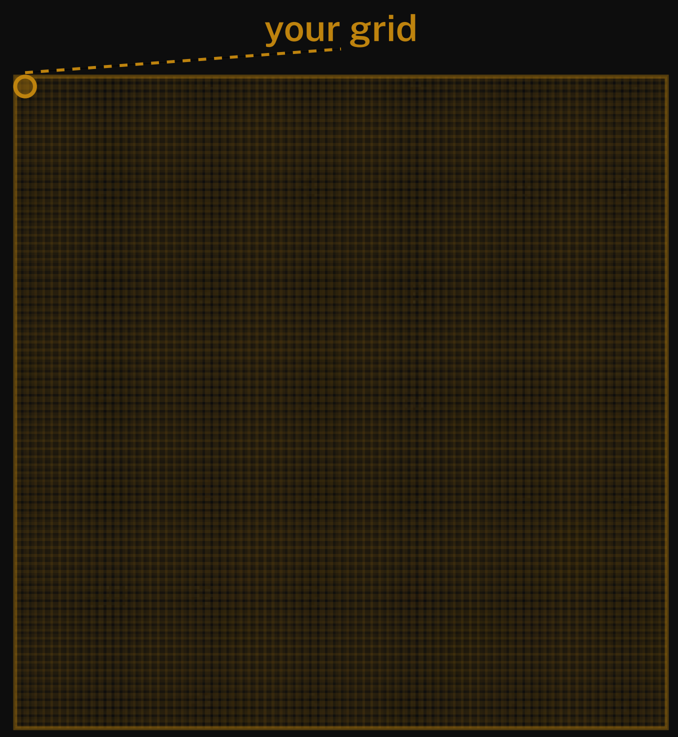
Green Eggs and Ham

70 tokens · 4,900 cells

Speaker notes

A whole Dr Seuss book is 70 distinct tokens, so ~4,900 cells---a sheet about 70cm square, call it a poster. And their grid is the little gold square in the corner. Note the vocabulary is what matters here, not the length: the book is a couple of thousand words, but Seuss reuses them relentlessly.

Frankenstein



Frankenstein

7,441 tokens · 55 million cells

the cells are now too small to draw

Speaker notes

The novel in the booklet: 7,441 distinct tokens, 55 million cells. The grid lines have stopped being drawable --- that grey is the grid. On paper it's about 5,500 square metres, most of a football field, and their model is the gold speck. This is also why the booklet doesn't print a grid: at this size you list only the cells that aren't empty.

a real vocabulary: **100,000** tokens, **10 billion** cells, **1 km^2** of paper

Speaker notes

Stop drawing and start measuring, because the picture gives out here. A commercial model's vocabulary is around 100,000, so its bigram grid alone is 10 billion cells---a square kilometre of paper, near enough the whole Acton campus. And that is still the toy version: one word of context, tally marks, none of the four fixes we just went through. Two more ballparks if you want them. The real thing: a frontier model's ~1.5 trillion parameters at 1cm each is about 150 km^2, a square 12km on a side, a hundred Acton campuses, twenty-odd Lake Burley Griffins. And time: if hand-training 100 tokens took you 10 minutes, the tens of trillions of tokens these are trained on would take about 5.7 million years at your rate---you'd have had to start around when our ancestors split from the chimpanzees to be finishing now. The punchline isn't any one figure. It's that the mechanism didn't change, only the dosage.

still just **tokens in → tokens out**

Speaker notes

Every reply from Claude or ChatGPT is sampled one word at a time, exactly like their dice rolls --- that's why "regenerate" gives a different answer. More context, learned where to look, billions of numbers instead of tallies, a round of manners --- but the loop is the one they ran by hand.

Questions

what new questions do you have about large language models?

What next?

how will this change the way you think about and use LLMs in the future?

Australian
National
University

School of
Cybernetics

www.llmsunplugged.org